

```
import tensorflow as tf
from tensorflow.keras import datasets, layers, models
from sklearn.model_selection import train_test_split
import matplotlib.pyplot as plt

#
# 1. Chargement et prétraitement des données CIFAR-10
#
(train_images, train_labels), (test_images, test_labels) = datasets.cifar10.load_data()

# Normalisation des pixels
train_images, test_images = train_images / 255.0, test_images / 255.0

# Séparation d'un jeu de validation à partir du jeu d'entraînement (20%)
train_images, val_images, train_labels, val_labels = train_test_split(
    train_images, train_labels, test_size=0.2, random_state=42
)
```

Downloading data from <https://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz>
 170498071/170498071 — 13s 0us/step

```
model = models.Sequential([
    # Bloc 1
    layers.Conv2D(32, (3,3), activation='relu', padding='same', input_shape=(32,32,3)),
    layers.Conv2D(32, (3,3), activation='relu', padding='same'),
    layers.MaxPooling2D((2,2)),
    layers.Dropout(0.25),

    # Bloc 2
    layers.Conv2D(64, (3,3), activation='relu', padding='same'),
    layers.Conv2D(64, (3,3), activation='relu', padding='same'),
    layers.MaxPooling2D((2,2)),
    layers.Dropout(0.25),

    # Bloc 3
    layers.Conv2D(128, (3,3), activation='relu', padding='same'),
    layers.Conv2D(128, (3,3), activation='relu', padding='same'),
    layers.MaxPooling2D((2,2)),
    layers.Dropout(0.25),

    # Classification
    layers.Flatten(),
    layers.Dense(512, activation='relu'),
    layers.Dropout(0.5),
    layers.Dense(10, activation='softmax')
])
```

/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input`
 super().__init__(activity_regularizer=activity_regularizer, **kwargs)

```
# 3. Compilation du modèle
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

# 4. Entraînement du modèle
history = model.fit(
    train_images, train_labels,
    epochs=20,
    batch_size=64,
    validation_data=(val_images, val_labels)
)

# 5. Évaluation finale sur le jeu de test
test_loss, test_acc = model.evaluate(test_images, test_labels)
print(f'Test accuracy: {test_acc:.4f}')

# 6. Visualisation des courbes d'apprentissage
#
# Accuracy
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.ylim([0, 1])
plt.legend(loc='lower right')
plt.title('Training and Validation Accuracy')
plt.show()
```

```
# Loss
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend(loc='upper right')
plt.title('Training and Validation Loss')
plt.show()
```

```

Epoch 1/20
625/625 ————— 20s 12ms/step - accuracy: 0.3617 - loss: 1.7176 - val_accuracy: 0.4962 - val_loss: 1.4037
Epoch 2/20
625/625 ————— 5s 8ms/step - accuracy: 0.5413 - loss: 1.2736 - val_accuracy: 0.6186 - val_loss: 1.0653
Epoch 3/20
625/625 ————— 5s 9ms/step - accuracy: 0.6201 - loss: 1.0785 - val_accuracy: 0.6765 - val_loss: 0.8977
Epoch 4/20
625/625 ————— 5s 8ms/step - accuracy: 0.6657 - loss: 0.9518 - val_accuracy: 0.7113 - val_loss: 0.8160
Epoch 5/20
625/625 ————— 5s 8ms/step - accuracy: 0.6963 - loss: 0.8602 - val_accuracy: 0.7222 - val_loss: 0.7937
Epoch 6/20
625/625 ————— 5s 9ms/step - accuracy: 0.7221 - loss: 0.7929 - val_accuracy: 0.7374 - val_loss: 0.7379
Epoch 7/20
625/625 ————— 5s 9ms/step - accuracy: 0.7415 - loss: 0.7392 - val_accuracy: 0.7576 - val_loss: 0.6922
Epoch 8/20
625/625 ————— 5s 9ms/step - accuracy: 0.7557 - loss: 0.7014 - val_accuracy: 0.7729 - val_loss: 0.6641
Epoch 9/20
625/625 ————— 5s 8ms/step - accuracy: 0.7640 - loss: 0.6705 - val_accuracy: 0.7786 - val_loss: 0.6438
Epoch 10/20
625/625 ————— 6s 9ms/step - accuracy: 0.7756 - loss: 0.6381 - val_accuracy: 0.7870 - val_loss: 0.6180
Epoch 11/20
625/625 ————— 5s 8ms/step - accuracy: 0.7863 - loss: 0.6084 - val_accuracy: 0.7709 - val_loss: 0.6764
Epoch 12/20
625/625 ————— 5s 8ms/step - accuracy: 0.7915 - loss: 0.5914 - val_accuracy: 0.7930 - val_loss: 0.6047
Epoch 13/20
625/625 ————— 6s 9ms/step - accuracy: 0.7991 - loss: 0.5707 - val_accuracy: 0.7853 - val_loss: 0.6119
Epoch 14/20
625/625 ————— 5s 8ms/step - accuracy: 0.8050 - loss: 0.5573 - val_accuracy: 0.7931 - val_loss: 0.6139
Epoch 15/20
625/625 ————— 6s 9ms/step - accuracy: 0.8090 - loss: 0.5476 - val_accuracy: 0.7983 - val_loss: 0.5949
Epoch 16/20
625/625 ————— 5s 8ms/step - accuracy: 0.8142 - loss: 0.5321 - val_accuracy: 0.7980 - val_loss: 0.6034
Epoch 17/20
625/625 ————— 6s 10ms/step - accuracy: 0.8161 - loss: 0.5203 - val_accuracy: 0.7945 - val_loss: 0.6123
Epoch 18/20
625/625 ————— 5s 8ms/step - accuracy: 0.8202 - loss: 0.5103 - val_accuracy: 0.8048 - val_loss: 0.5878
Epoch 19/20
625/625 ————— 6s 10ms/step - accuracy: 0.8242 - loss: 0.5039 - val_accuracy: 0.8025 - val_loss: 0.6002

```